

Ejercicios Propuestos #2

Parte 1: Manejo de datos con Pandas

Ejercicio 1

Escribir programa que genere y muestre por pantalla un DataFrame con los datos de la tabla siguiente. Además adicione una variable adicional llamada *balance* (ventas - gastos).

Mes	Ventas	Gastos
Enero	30500	22000
Febrero	35600	23400
Marzo	28300	18100
Abril	33900	20700

```
In [46]: import pandas as pd

# forma 1
datos = {'Mes':['Enero', 'Febrero', 'Marzo', 'Abril'], 'Ventas':[30500, 35600, 28300, 33900], 'Gastos':[22000, 23400, 18100, 20700]}
contabilidad = pd.DataFrame(datos)
contabilidad['Balance'] = contabilidad['Ventas'] - contabilidad['Gastos']
display(contabilidad)

print('-----')
# forma 2
datos = [['Enero', 30500, 22000], ['Febrero', 35600, 23400], ['Marzo', 28300, 18100], ['Abril', 33900, 20700]]
contabilidad = pd.DataFrame(datos, columns=['Mes', 'Ventas', 'Gastos'])
contabilidad['Balance'] = contabilidad['Ventas'] - contabilidad['Gastos']
display(contabilidad)
```

	Mes	Ventas	Gastos	Balance
0	Enero	30500	22000	8500
1	Febrero	35600	23400	12200
2	Marzo	28300	18100	10200
3	Abril	33900	20700	13200

	Mes	Ventas	Gastos	Balance
0	Enero	30500	22000	8500
1	Febrero	35600	23400	12200
2	Marzo	28300	18100	10200
3	Abril	33900	20700	13200

Ejercicio 2

Escribir un programa que solicite al usuario ingresar por pantalla las ventas de un rango de años (usando como parámetros el año de inicio y el año de fin) y muestre por pantalla una serie con los datos de las ventas indexado por los años, antes y después de aplicarles un descuento del 10%. Debe controlar los errores por si lo ingresado no son números.

Ejemplo de ingreso de datos:

- Introduce el año inicial: 2010
- Introduce el año final: 2015
- Introduce las ventas del año 2010: 100000
- Introduce las ventas del año 2011: 120000
- Introduce las ventas del año 2012: 115000
- Introduce las ventas del año 2013: 125000
- Introduce las ventas del año 2014: 130000
- Introduce las ventas del año 2015: 122000

```
In [47]: import pandas as pd

try:
    años = []
    valores = []
    inicio= int(input('Introduce el año inicial:'))
    fin= int(input('Introduce el año final:'))
    if inicio > fin:
        print('El año de inicio debe ser mayor o igual al año de final')
    else:
        for a in range(inicio,fin+1):
            años.append(a)
            valores.append(float(input('Introduce las ventas del año '+str(a)+' :')))
        data = pd.DataFrame({'Años':años, 'Ventas':valores})
        data['Con descuento'] = data['Ventas']*0.9
        display(data)
except:
    print('Debe ingresar valores numéricos.')
```

```
Introduce el año inicial:2010
Introduce el año final:2015
Introduce las ventas del año 2010:10000
Introduce las ventas del año 2011:12000
Introduce las ventas del año 2012:11500
Introduce las ventas del año 2013:12500
Introduce las ventas del año 2014:13000
Introduce las ventas del año 2015:12200
```

	Años	Ventas	Con descuento
0	2010	10000.0	9000.0
1	2011	12000.0	10800.0
2	2012	11500.0	10350.0
3	2013	12500.0	11250.0
4	2014	13000.0	11700.0
5	2015	12200.0	10980.0

Ejercicio 3

a. Escribir una función que reciba un diccionario con las notas de los alumnos en curso en un examen y devuelva una serie con los principales estadísticos de las notas.

```
In [48]: notas = {'Juan':18, 'María':13, 'Pedro':8, 'Carmen': 17, 'Luis': 15, 'Sara': 12, 'Francisco': 5,
'Jorge': 18, 'Ana': 10}

def estadisticos(dict):
    data = pd.Series(dict)
    print(data.describe())

estadisticos(notas)
```

count	9.000000
mean	12.888889
std	4.594683
min	5.000000
25%	10.000000
50%	13.000000
75%	17.000000
max	18.000000
dtype:	float64

b. Escribir otra función que reciba el diccionario anterior y devuelva una serie con las notas de los alumnos aprobados ordenadas de mayor a menor (considere aprobado ≥ 11)

```
In [49]: def aprobados(dict):
    data = pd.Series(dict)
    print(data[data >= 11].sort_values(ascending = False))

aprobados(notas)
```

Juan	18
Jorge	18
Carmen	17
Luis	15
María	13
Sara	12
dtype:	int64

Ejercicio 4

El fichero **cotizacion.csv** contiene las cotizaciones de las empresas del IBEX35 con las siguientes columnas: nombre (nombre de la empresa), Final (precio de la acción al cierre de bolsa), Máximo (precio máximo de la acción durante la jornada), Mínimo (precio mínimo de la acción durante la jornada), volumen (Volumen al cierre de bolsa), Efectivo (capitalización al cierre en miles de euros).

a. Cargar este fichero en un DataFrame. Tenga en cuenta que debe seleccionar adecuadamente el separador del fichero (sep), el separador decimal (decimal) y el separador de miles (thousand).

b. A partir del DataFrame cargado, debe mostrar otro con el mínimo, el máximo y la media de cada columna. Debe quedar algo así:

	Final	Máximo	Mínimo	Volumen	Efectivo
Mínimo	1.016500	4.067500	1.016500	1.221000e+03	2343.090
Máximo	19705.000000	19875.000000	19675.000000	3.612969e+07	145765.440
Media	2796.768757	3170.113357	3136.510471	4.252279e+06	31767.778

```
In [50]: def resumen_cotizaciones(fichero):
    df = pd.read_csv(fichero, sep=';', decimal=',', thousands='.', index_col=0)
    return pd.DataFrame([df.min(), df.max(), df.mean()], index=['Mínimo', 'Máximo', 'Media'])

resumen_cotizaciones('data/cotizacion.csv')
```

Out[50]:

	Final	Máximo	Mínimo	Volumen	Efectivo
Mínimo	1.016500	4.067500	1.016500	1.221000e+03	2343.090
Máximo	19705.000000	19875.000000	19675.000000	3.612969e+07	145765.440
Media	2796.768757	3170.113357	3136.510471	4.252279e+06	31767.778

Ejercicio 5

El archivo **titanic.csv** contiene información sobre los pasajeros del Titanic. Cargar el fichero en un dataframe. Realice las siguientes actividades:

1. Mostrar las dimensiones del DataFrame, el número de datos que contiene, los nombres de sus columnas y filas, los tipos de datos de las columnas, las 10 primeras filas y las 10 últimas filas.
2. Mostrar los datos del pasajero con identificador 148 (usando **loc**).
3. Mostrar las filas pares del DataFrame (usando **iloc** y opcionalmente **range** para obtener las posiciones pares).
4. Mostrar los nombres de las personas que iban en primera clase ordenadas alfabéticamente (usando **sort_values**).
5. Mostrar el porcentaje de personas que sobrevivieron y murieron (usando **value_counts(normalize = True)**).
6. Mostrar el porcentaje de personas que sobrevivieron en cada clase (usando **groupby** y **value_counts(normalize = True)**).
7. Eliminar del DataFrame los pasajeros con edad desconocida (usando **dropna** sobre el campo de edad).
8. Mostrar la edad media de las mujeres que viajaban en cada clase (usando **groupby**, **mean** y **unstack**).
9. Añadir una nueva columna booleana para ver si el pasajero era menor de edad o no.
10. Mostrar por pantalla el porcentaje de menores y mayores de edad que sobrevivieron en cada clase (usando **groupby** y **value_counts(normalize = True)**).

```
In [51]: import pandas as pd

# Pregunta 1
titanic = pd.read_csv('data/titanic.csv', index_col=0)
print('Dimensiones:', titanic.shape)
print('-----')
print('Número de elemntos:', titanic.size)
print('-----')
print('Nombres de columnas:', titanic.columns)
print('-----')
print('Nombres de filas:', titanic.index)
print('-----')
print('Tipos de datos:\n', titanic.dtypes)
print('-----')
display('Primeras 10 filas:\n', titanic.head(10))
print('-----')
display('Últimas 10 filas:\n', titanic.tail(10))
```

```
Dimensiones: (891, 11)
-----
Número de elemntos: 9801
-----
Nombres de columnas: Index(['Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket',
                             'Fare', 'Cabin', 'Embarked'],
                             dtype='object')
-----
Nombres de filas: Int64Index([ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
                               ...
                               882, 883, 884, 885, 886, 887, 888, 889, 890, 891],
                               dtype='int64', name='PassengerId', length=891)
-----
Tipos de datos:
Survived      int64
Pclass        int64
Name          object
Sex           object
Age          float64
SibSp         int64
Parch         int64
Ticket        object
Fare         float64
Cabin         object
Embarked      object
dtype: object
-----
```

'Primeras 10 filas:\n'

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C

'Últimas 10 filas:\n'

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
882	0	3	Markun, Mr. Johann	male	33.0	0	0	349257	7.8958	NaN	S
883	0	3	Dahlberg, Miss. Gerda Ulrika	female	22.0	0	0	7552	10.5167	NaN	S
884	0	2	Banfield, Mr. Frederick James	male	28.0	0	0	C.A./SOTON 34068	10.5000	NaN	S
885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SOTON/OQ 392076	7.0500	NaN	S
886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29.1250	NaN	Q
887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

```
In [52]: # Pregunta 2
titanic.loc[[148]]
```

Out[52]:

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
148	0	3	Ford, Miss. Robina Maggie "Ruby"	female	9.0	2	2	W./C. 6608	34.375	NaN	S

```
In [53]: # Pregunta 3
titanic.iloc[range(0,titanic.shape[0],2)]
```

Out[53]:

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
883	0	3	Dahlberg, Miss. Gerda Ulrika	female	22.0	0	0	7552	10.5167	NaN	S
885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SOTON/OQ 392076	7.0500	NaN	S
887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

446 rows × 11 columns

```
In [54]: # Pregunta 4
titanic[titanic["Pclass"]==1]['Name'].sort_values()
```

```
Out[54]: PassengerId
731      Allen, Miss. Elisabeth Walton
306      Allison, Master. Hudson Trevor
298      Allison, Miss. Helen Loraine
499      Allison, Mrs. Hudson J C (Bessie Waldo Daniels)
461      Anderson, Mr. Harry
...
156      Williams, Mr. Charles Duane
352      Williams-Lambert, Mr. Fletcher Fellows
56      Woolner, Mr. Hugh
556      Wright, Mr. George
326      Young, Miss. Marie Grice
Name: Name, Length: 216, dtype: object
```

```
In [55]: # Pregunta 5
print(titanic['Survived'].value_counts()/titanic['Survived'].count() * 100)

# Otra forma
print(titanic['Survived'].value_counts(normalize=True) * 100)

0    61.616162
1    38.383838
Name: Survived, dtype: float64
0    61.616162
1    38.383838
Name: Survived, dtype: float64
```

```
In [56]: # Pregunta 6
titanic.groupby('Pclass')['Survived'].value_counts(normalize=True)
```

```
Out[56]: Pclass  Survived
1          1          0.629630
          0          0.370370
2          0          0.527174
          1          0.472826
3          0          0.757637
          1          0.242363
Name: Survived, dtype: float64
```



```
In [57]: # Pregunta 7
titanic.dropna(subset=['Age'])

# Alternativa
# titanic = titanic[titanic['Age'].notna()]
```

Out[57]:

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
PassengerId											
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...
886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29.1250	NaN	Q
887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

714 rows × 11 columns

```
In [58]: # Pregunta 8
print(titanic.groupby(['Pclass', 'Sex'])['Age'].mean().unstack()['female'])

Pclass
1    34.611765
2    28.722973
3    21.750000
Name: female, dtype: float64
```

```
In [59]: # Pregunta 9
titanic['Young'] = titanic['Age'] < 18
titanic.head()
```

Out[59]:

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Young
PassengerId												
1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	False
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C	False
3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	False
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S	False
5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S	False

```
In [60]: # Pregunta 10
titanic.groupby(['Pclass', 'Young'])['Survived'].value_counts(normalize = True) * 100
```

```
Out[60]: Pclass  Young  Survived
1         False    1         61.274510
          0         38.725490
          True     1         91.666667
          0         8.333333
2         False    0         59.006211
          1         40.993789
          True     1         91.304348
          0         8.695652
3         False    0         78.208232
          1         21.791768
          True     0         62.820513
          1         37.179487
Name: Survived, dtype: float64
```

Ejercicio 6

El siguiente código carga el conjunto de datos olímpicos (olympics.csv), que se obtuvo de la entrada de Wikipedia en [All Time Olympic Games Medals](https://en.wikipedia.org/wiki/All-time_Olympic_Games_medal_table) (https://en.wikipedia.org/wiki/All-time_Olympic_Games_medal_table), y hace algunos comandos básicos de limpieza de datos. Las columnas están organizadas como # de juegos de verano, medallas de verano, # de juegos de invierno, medallas de invierno, # total de juegos, # total de medallas. A partir de la siguiente tabla generada del siguiente código, y usando las librerías **Pandas** y **NumPy** elabore el código que permita responder las siguientes preguntas:

```
In [61]: import pandas as pd
import numpy as np

df = pd.read_csv('data/olympics.csv', index_col=0, skiprows=1)
for col in df.columns:
    if col[:2]=='01':
        df.rename(columns={col:'Gold'+col[4:]}, inplace=True)
    if col[:2]=='02':
        df.rename(columns={col:'Silver'+col[4:]}, inplace=True)
    if col[:2]=='03':
        df.rename(columns={col:'Bronze'+col[4:]}, inplace=True)
    if col[:1]=='#':
        df.rename(columns={col:'#'+col[1:]}, inplace=True)

names_ids = df.index.str.split('\s\(') # split the index by '('

df.index = names_ids.str[0] # the [0] element is the country name (new index)
df['ID'] = names_ids.str[1].str[:3] # the [1] element is the abbreviation or ID (take first 3 char
acters from that)

df = df.drop('Totals')

df.head(10)
```

Out[61]:

	# Summer	Gold	Silver	Bronze	Total	# Winter	Gold.1	Silver.1	Bronze.1	Total.1	# Games	Gold.2	Silver.2	B
Afghanistan	13	0	0	2	2	0	0	0	0	0	13	0	0	
Algeria	12	5	2	8	15	3	0	0	0	0	15	5	2	
Argentina	23	18	24	28	70	18	0	0	0	0	41	18	24	
Armenia	5	1	2	9	12	6	0	0	0	0	11	1	2	
Australasia	2	3	4	5	12	0	0	0	0	0	2	3	4	
Australia	25	139	152	177	468	18	5	3	4	12	43	144	155	
Austria	26	18	33	35	86	22	59	78	81	218	48	77	111	
Azerbaijan	5	6	5	15	26	5	0	0	0	0	10	6	5	
Bahamas	15	5	2	5	12	0	0	0	0	0	15	5	2	
Bahrain	8	0	0	1	1	0	0	0	0	0	8	0	0	

```
In [62]: df.head()
names_ids = df.index.str.split('\s\(')
names_ids
```

```
Out[62]: Index([
    ['Afghanistan'],
    ['Algeria'],
    ['Argentina'],
    ['Armenia'],
    ['Australasia'],
    ['Australia'],
    ['Austria'],
    ['Azerbaijan'],
    ['Bahamas'],
    ['Bahrain'],
    ...
    ['Uruguay'],
    ['Uzbekistan'],
    ['Venezuela'],
    ['Vietnam'],
    ['Virgin Islands'],
    ['Yugoslavia'],
    ['Independent Olympic Participants'],
    ['Zambia'],
    ['Zimbabwe'],
    ['Mixed team']],
dtype='object', length=146)
```

El país ha ganado más medallas de oro en los juegos de verano se obtiene del siguiente código:

```
In [63]: # Forma 1
np.argmax(df['Gold'])

# Forma 2
df[df['Gold']==np.max(df['Gold'])].index.tolist()[0]
```

```
Out[63]: 'United States'
```

a. ¿Qué país ha ganado más medallas de oro en los juegos olímpicos de invierno?

```
In [64]: df[df['Gold.1'] == np.max(df['Gold.1'])].index.tolist()[0], np.max(df['Gold.1'])

Out[64]: ('Norway', 118)
```

b. ¿Qué país ha ganado más medallas de plata en los juegos de verano?

```
In [65]: df[df['Silver'] == np.max(df['Silver'])].index.tolist()[0], np.max(df['Silver'])

Out[65]: ('United States', 757)
```

c. ¿Qué país tuvo la mayor diferencia entre el conteo de sus medallas de oro en verano e invierno?

```
In [66]: abs(df['Gold'] - df['Gold.1']).sort_values(ascending=False).head(1).sort_index()

Out[66]: United States      880
dtype: int64
```

d. Escribe una función para crear una Serie llamada "Points", la cual es un valor ponderado, donde cada medalla de oro (Gold.2) cuenta con 3 puntos, medalla de plata (Silver.2) con 2 puntos y medalla de bronce (Bronze.2) por 1 punto. La función debe devolver solo la columna (un objeto Serie) que se creó.

Esta función debería devolver una Serie llamada Points de longitud 146

```
In [67]: Points = df['Gold.2']*3 + df['Silver.2']*2 + df['Bronze.2']
```

e. ¿Qué país pertenecen al top 10 de la serie "Points"? ¿Cuál es la diferencia entre el mayor y menor valor de la serie "Points"? ¿Cuál es el valor promedio de esta serie?

```
In [68]: Points.sort_values(ascending = False).head(10)
```

```
Out[68]: United States    5684
         Soviet Union    2526
         Great Britain    1574
         Germany         1546
         France          1500
         Italy            1333
         Sweden          1217
         China           1120
         East Germany     1068
         Russia           1042
         dtype: int64
```

f. ¿Qué país tiene la mayor diferencia entre el conteo de sus medallas de oro de verano y el conteo de sus medallas de oro de invierno en relación con al conteo total de sus medallas de oro?

$$\frac{\text{Summer Gold} - \text{Winter Gold}}{\text{Total Gold}}$$

Sólo incluye países que hayan ganado al menos 1 medalla de oro tanto en verano como en invierno.

```
In [70]: df_gold = df.loc[(df['Gold']>=1) & (df['Gold.1']>=1)]
         serie_silver = abs(df_gold['Gold'] - df_gold['Gold.1'])/df_gold['Gold.2']
         serie_silver.sort_values(ascending = True).head(1)
```

```
Out[70]: Canada    0.024793
         dtype: float64
```

Parte 2: Clases y Objetos

Ejercicio 1

a. Cree una clase llamada **Gato**, la cual debe cotener una variable llamada tipo y cuyo valor sea 'felino'. La clase debe permitir ser instanciada con dos valores (obligatorios), el primero que corresponda al nombre del gato y el segundo al color del pelo. La clase debe tener un cosnstructor (`__init__`).

```
In [1]: class Gato:
         def __init__(self,nombre,pelo):
             self.nombre = nombre
             self.pelo = pelo

         tipo="felino"
         descripcion="Es un gato"
```

b. Cree dos objetos del tipo Gato. El primer objeto se llamara gato1, el nombre del gato es "Garfield" y el color de pelo es "naranja". El segundo objeto se llama gato2, el nombre del gato es "Silvestre" y el color del pelo es "gris". Muestre en pantalla los principales atributos y métodos de cada Gato instanciado.

```
In [2]: gato1= Gato("Garfield","naranja")
        gato2= Gato("Silvestre","gris")

        print(gato1.nombre)
        print(gato1.pelo)
        print(gato1.tipo)
        print(gato1.descripcion)
        print('\n')
        print(gato2.nombre)
        print(gato2.pelo)
        print(gato2.tipo)
        print(gato2.descripcion)
```

```
Garfield
naranja
felino
Es un gato
```

```
Silvestre
gris
felino
Es un gato
```

Ejercicio 2

Crear la clase Vendedor que permita:

- Añadir nuevos vendedores (idVendedor, nombre).
- Modificar unidades vendidas (Todos empiezan en 0, pero debe permitir añadir las nuevas ventas).
- Calcular el porcentaje de avance (2 decimales) con respecto a su meta (Todos tienen una meta de 1450 unidades): El vendedor Alfonso está al 48.52% de avance.

```
In [8]: class Vendedor:

        # Las ventas son cero por defecto
        def __init__(self, idVendedor, nombre, ventas = 0):
            self.idVendedor = idVendedor
            self.nombre = nombre
            self.ventas = 0

        def __str__(self):
            return "IdVendedor: "+self.idVendedor + " \nNombre " + self.nombre+ \
                " \nVentas: " + str(self.ventas) + " soles."

        def registrarVentas(self, new_ventas):
            self.ventas = new_ventas

        def calcularAvance(self):
            avance = round(self.ventas*100/1450,2)
            print('El vendedor ', self.nombre, ' está al ', avance, '% de avance.' )
```

```
In [9]: Alfonso = Vendedor('0001', 'Alfonso')
        print(Alfonso)
```

```
IdVendedor: 0001
Nombre Alfonso
Ventas: 0 soles.
```

```
In [10]: Alfonso.calcularAvance()
```

```
El vendedor  Alfonso  está al  0.0 % de avance.
```

```
In [11]: Alfonso.registrarVentas(703.54)
```

```
In [12]: print(Alfonso)
```

```
IdVendedor: 0001
Nombre Alfonso
Ventas: 703.54 soles.
```

```
In [13]: Alfonso.calcularAvance()
```

El vendedor Alfonso está al 48.52 % de avance.

Ejercicio 3

Observe y ejecute el siguiente código, con el que tenemos la posibilidad de implementar la clase hoteles, con los atributos de los tipos correspondientes, y de mostrar los hoteles según lo solicitado (cada vez que adicione algo vuelva a ejecutarlo para que se actualice):

```
In [3]: class Hotel(object):
        """ Hotel: sus atributos son: nombre, ubicacion, puntaje y precio."""

        def __init__(self, nombre = '*', ubicacion = '*',
                        puntaje = 0, precio = float("inf")):
            """ nombre y ubicacion deben ser cadenas no vacías,
                puntaje y precio son números.
                Si el precio es 0 se reemplaza por infinito. """

            if len(nombre) != 0:
                self.nombre = nombre
            else:
                raise TypeError ("El nombre debe ser una cadena no vacía")

            if len(ubicacion) != 0:
                self.ubicacion = ubicacion
            else:
                raise TypeError ("La ubicación debe ser una cadena no vacía")

            if puntaje != 0:
                self.puntaje = puntaje
            else:
                raise TypeError ("El puntaje debe ser un número")

            if precio != 0:
                self.precio = precio
            else:
                self.precio = float("inf")

        # método "mágico" para mostrar el contenido de un hotel usando print

        def __str__(self):
            """ Muestra el hotel según lo requerido. """
            return self.nombre + " de " + self.ubicacion + \
                " - Puntaje: " + str(self.puntaje) + " - Precio: " + \
                str(self.precio) + " soles."

        # método general para mostrar el contenido de un hotel

        def mostrar(self):
            return self.__str__()

        #a partir de aquí deben crearse Los métodos solicitados en los ejercicios siguientes

        def ratio(self):
            """ Calcula la relación calidad-precio de un hotel de acuerdo
                a la fórmula que nos dio el cliente. """
            return ((self.puntaje**2)*10.)/self.precio

        def __gt__(self, otro):
            return self.ratio() > otro.ratio()

        def __lt__(self, otro):
            return self.ratio() < otro.ratio()

        def __ge__(self, otro):
            return self.ratio() >= otro.ratio()

        def __le__(self, otro):
            return self.ratio() <= otro.ratio()

        def __eq__(self, otro):
            return self.ratio() == otro.ratio()

        def __ne__(self, otro):
            return self.ratio() != otro.ratio()
```

a. Instancie el hotel Sheraton de Lima, cuyo puntaje es de 3.25 y el precio es de 200 soles la habitación. Muestre los resultados de este hotel usando la función **print** (el método "mágico" **str** permite esto)

```
In [4]: Hotel1= Hotel('Hotel Sheraton','Lima',3.25,200)

print(Hotel1)
```

Hotel Sheraton de Lima - Puntaje: 3.25 - Precio: 200 soles.

b. Haga prueba de la función **mostrar** implementado (es la segunda forma de mostrar el contenido de un Hotel sin usar **print**)

```
In [5]: Hotel1.mostrar()
```

```
Out[5]: 'Hotel Sheraton de Lima - Puntaje: 3.25 - Precio: 200 soles.'
```

c. Para resolver las comparaciones entre hoteles, será necesario definir algunos métodos especiales que permiten comparar objetos. Primero un método auxiliar en la clase Hotel: **ratio(self)** que calcula la relación calidad-precio de una instancia de **Hotel** según la fórmula que se indica a continuación:

```
def ratio(self): """ Calcula la relación calidad-precio de un hotel de acuerdo a la fórmula que nos dio el cliente. """ return
((self.puntaje**2)*10.)/self.precio
```

d. Implemente métodos "mágicos" de comparación de ratios en la clase Hotel. Estos métodos permitirán comparar objetos de la misma clase con operadores lógicos tradicionales.

#implemente los siguientes métodos "mágicos" en la clase Hotel. Ejemplo: def __gt__(self, otro): #mayor que return self.ratio() > otro.ratio()
#implemente __lt__ menor que #implemente __ge__ mayor o igual que #implemente __le__ menor o igual que #implemente __eq__ igual
que #implemente __ne__ no es igual que

e. Luego compare el ratio del hotel Sheraton (vuelva a instanciarlo) con el ratio del hotel Libertadores del Cuzco, cuyo puntaje es de 6 y el precio es de 150 soles la habitación (use todas las funciones creadas).

```
In [6]: # Completa el ejercicio de Las comparaciones aquí,
# utilizando operadores >, >=, <, <=, == y !=
h= Hotel('Hotel Sheraton','Lima',3.25,200)
i= Hotel('Hotel Libertadores','Cuzco',6.00,150)

print(h > i)
print(h >= i)
print(h < i)
print(h <= i)
print(h == i)
print(h != i)
```

```
False
False
True
True
False
True
```

f. Además de los hoteles anteriores (llamelos h1 y h2), instancie 3 hoteles más de su preferencia (que tengan atributos diferentes) y llamelos h3, h4 y h5.


```
In [7]: h1= Hotel('Hotel Sheraton','Lima',3.25,200)
h2= Hotel('Hotel Libertadores','Cuzco',6.00,150)
h3= Hotel('Hotel Madison','Lima',5.00,250)
h4= Hotel('Hotel Costa del Sol','Cajamarca',4.50,120)
h5= Hotel('Hotel San Agustin','Cuzco',5.00,180)

lista = [ h1, h2, h3, h4, h5 ]
lista.sort()
for hotel in lista:
    print(hotel)
# explique el criterio de la salida de este bucle

# mostrando los ratios de los hoteles
for hotel in lista:
    print("ratio", hotel.nombre, hotel.ratio())
```

```
Hotel Sheraton de Lima - Puntaje: 3.25 - Precio: 200 soles.
Hotel Madison de Lima - Puntaje: 5.0 - Precio: 250 soles.
Hotel San Agustin de Cuzco - Puntaje: 5.0 - Precio: 180 soles.
Hotel Costa del Sol de Cajamarca - Puntaje: 4.5 - Precio: 120 soles.
Hotel Libertadores de Cuzco - Puntaje: 6.0 - Precio: 150 soles.
ratio Hotel Sheraton 0.528125
ratio Hotel Madison 1.0
ratio Hotel San Agustin 1.3888888888888888
ratio Hotel Costa del Sol 1.6875
ratio Hotel Libertadores 2.4
```

g. Finalmente responda las siguientes preguntas:

- ¿Qué realiza la función **sort()**? ¿Qué criterio utiliza para ordenar los hoteles?
- ¿Qué es lo que realizan los bucles **for**?
- ¿Cuáles son los hoteles con el mejor y peor ratio calidad-precio?

Responda las preguntas en esta sección...

- **sort()** ordena la lista de hoteles de forma ascendente por defecto, el criterio de ordenamiento es por el ratio que calcula la relación calidad-precio de los hoteles.
- Con el bucle **for** se muestran los elementos de la lista de hoteles, en este caso haciendo uso de la función **str** (o mostrar) de la clase **Hotel()**.
- El hotel Libertadores de Cuzco (h2) es el que tiene el mejor ratio calidad - precio, mientras que el hotel Sheraton de Lima (h1) tiene el peor ratio.

Parte 3: Representaciones Gráficas con Python

La data a utilizar para la siguiente sección muestra el valor de las acciones de ciertas compañías a través de cierto periodo de tiempo. La data se encuentra con los otros archivos de la sesión 4, y es el archivo: **data_final_fpp.csv**

Para cargar el conjunto de datos y que al utilizar la función **prices.head()** se vea de la siguiente forma, podemos usar la siguiente sentencia:

```
prices = pd.read_csv("data_final_fpp.csv",index_col=0,parse_dates=True)
```

	2005-01-03	2005-01-04	2005-01-05	2005-01-06	2005-01-07	2005-01-10	2005-01-11	2005-01-12	2005-01-13	2005-01-14	...
INTC	23.07	22.61	22.39	22.46	22.80	22.88	22.54	23.16	22.82	23.02	...
MSFT	26.74	26.84	26.78	26.75	26.67	26.80	26.73	26.78	26.27	26.12	...
IBM	97.75	96.70	96.50	96.20	95.78	95.68	95.00	95.21	94.45	94.10	...
AAPL	4.52	4.57	4.61	4.61	4.95	4.93	4.61	4.68	4.99	5.01	...
AMZN	44.52	42.14	41.77	41.05	42.32	41.84	41.64	42.30	42.60	44.55	...

```
In [14]: import pandas as pd
```

```
prices = pd.read_csv("data/data_final_fpp.csv", index_col=0, parse_dates=True)
prices.head()
```

Out[14]:

	2005-01-03	2005-01-04	2005-01-05	2005-01-06	2005-01-07	2005-01-10	2005-01-11	2005-01-12	2005-01-13	2005-01-14	...	2017-06-14	2017-06-15	2017-06-16	2017-06-19	2017-06-20
INTC	23.07	22.61	22.39	22.46	22.80	22.88	22.54	23.16	22.82	23.02	...	35.53	35.31	35.21	35.51	34.86
MSFT	26.74	26.84	26.78	26.75	26.67	26.80	26.73	26.78	26.27	26.12	...	70.27	69.90	70.00	70.87	69.91
IBM	97.75	96.70	96.50	96.20	95.78	95.68	95.00	95.21	94.45	94.10	...	153.81	154.22	155.38	154.84	154.95
AAPL	4.52	4.57	4.61	4.61	4.95	4.93	4.61	4.68	4.99	5.01	...	145.16	144.29	142.27	146.34	145.01
AMZN	44.52	42.14	41.77	41.05	42.32	41.84	41.64	42.30	42.60	44.55	...	976.47	964.17	987.71	995.17	992.59

5 rows × 3143 columns

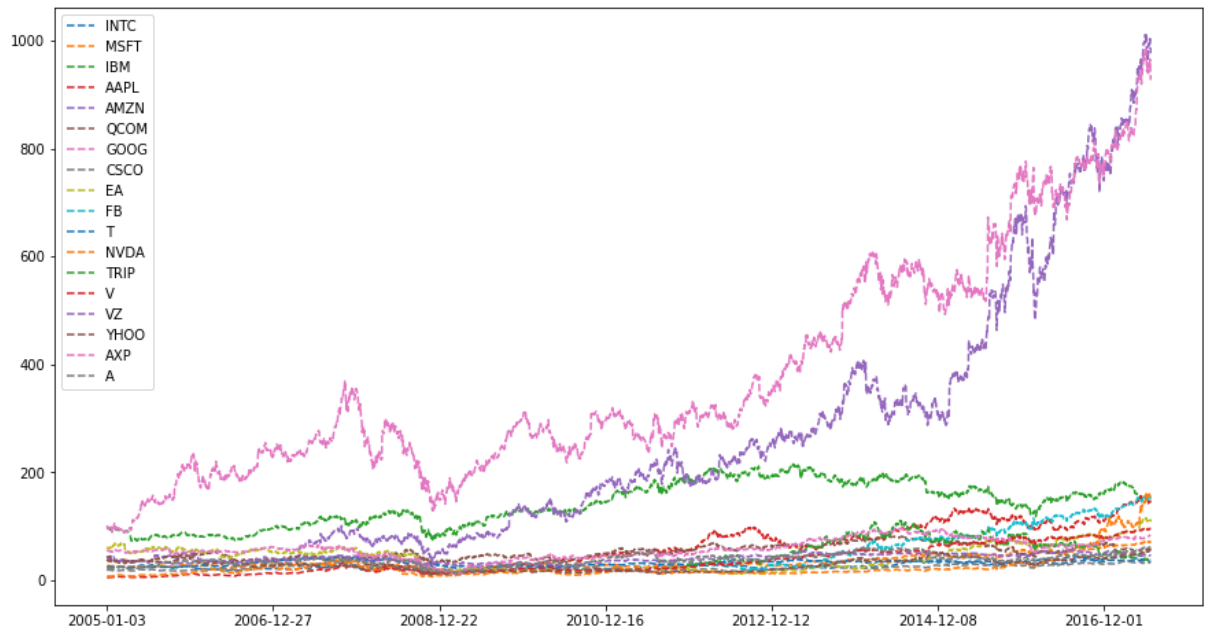


Ejercicio 1

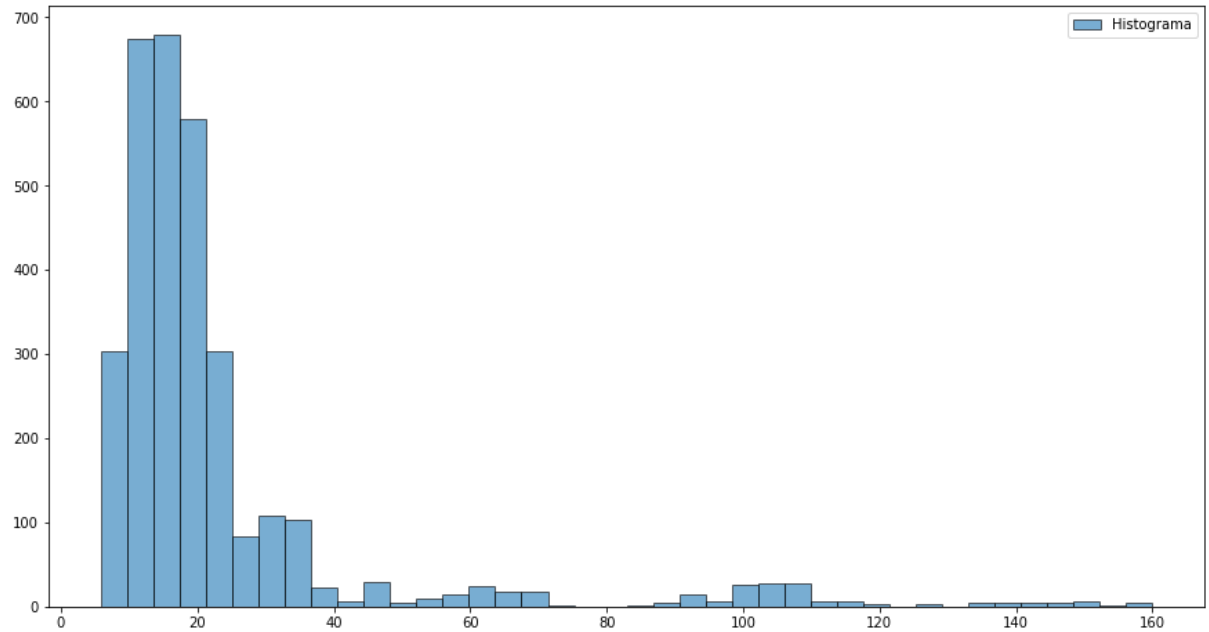
Realice los siguientes gráficos usando Series del dataframe cargado, y utilizando la **librería matplotlib.pyplot**: gráfico de línea, histograma, diagrama de caja y diagrama de dispersión. No se olvide de transponer previamente el dataframe antes de realizar los gráficos. Personalice los gráficos (colores, formas, etiquetas, leyendas, textos, entre otros)

```
In [25]: import numpy as np
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (15,8) # Aumentar el tamaño de la ventana
#gráfico de Línea
nuevo_price=prices.transpose()
nuevo_price.head()
nuevo_price.plot(kind='line',linestyle='--' )
plt.show()
```



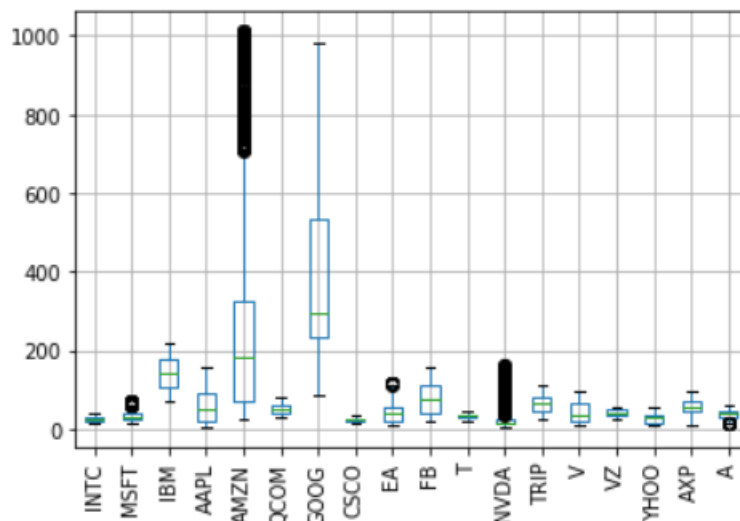
```
In [30]: #plt.hist(nuevo_price, bins = 20, alpha = 0.5, Label = 'Hist N2')
plt.hist(nuevo_price['NVDA'],
        bins = 40,
        alpha = 0.6,
        edgecolor = 'k',
        label = "Histograma")
plt.legend()
plt.show()
```



Ejercicio 2

Complete las sentencias necesarias para crear y representar lo más similar posible el boxplot siguiente. Ingrese el código completo en el espacio debajo.

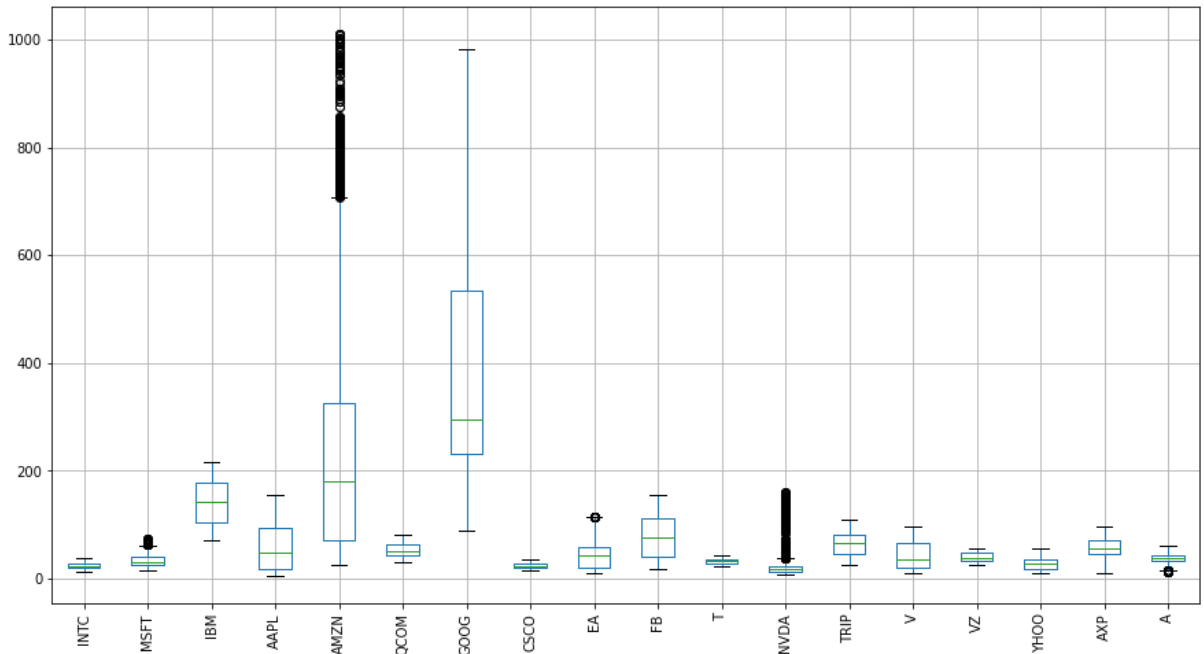
```
1 import matplotlib.pyplot as plt
2 transposePrices = prices.transpose()
3 _____.plot(_____)
4 plt.xticks(_____) # Colocamos las etiquetas rotadas 90°
5 _____ # Activa cuadrícula del gráfico
6 _____ # Finalmente mostramos el gráfico
```



```
In [50]: import matplotlib.pyplot as plt
transposePrices = prices.transpose()
transposePrices.plot(kind='box')
plt.xticks(rotation = 90) # Colocamos las etiquetas rotadas 90°
plt.grid() # Activa cuadrícula del gráfico
plt.show() # Finalmente mostramos el gráfico
```

C:\Users\Luis Felipe\Anaconda3\lib\site-packages\matplotlib\cbook__init__.py:1376: VisibleDeprecationWarning: Creating an ndarray from ragged nested sequences (which is a list-or-tuple of lists-or-tuples-or ndarrays with different lengths or shapes) is deprecated. If you meant to do this, you must specify 'dtype=object' when creating the ndarray.

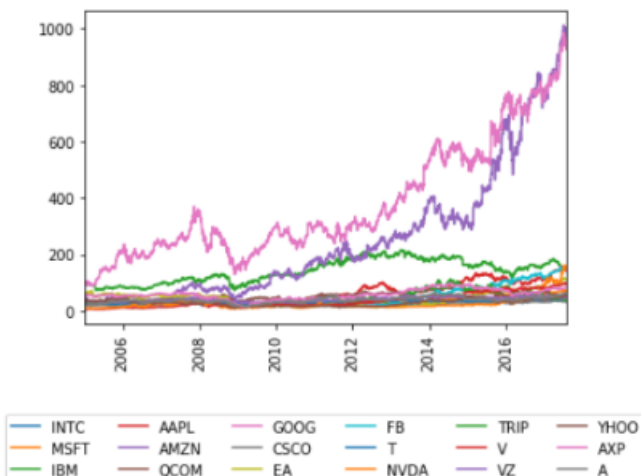
```
X = np.atleast_1d(X.T if isinstance(X, np.ndarray) else np.asarray(X))
```



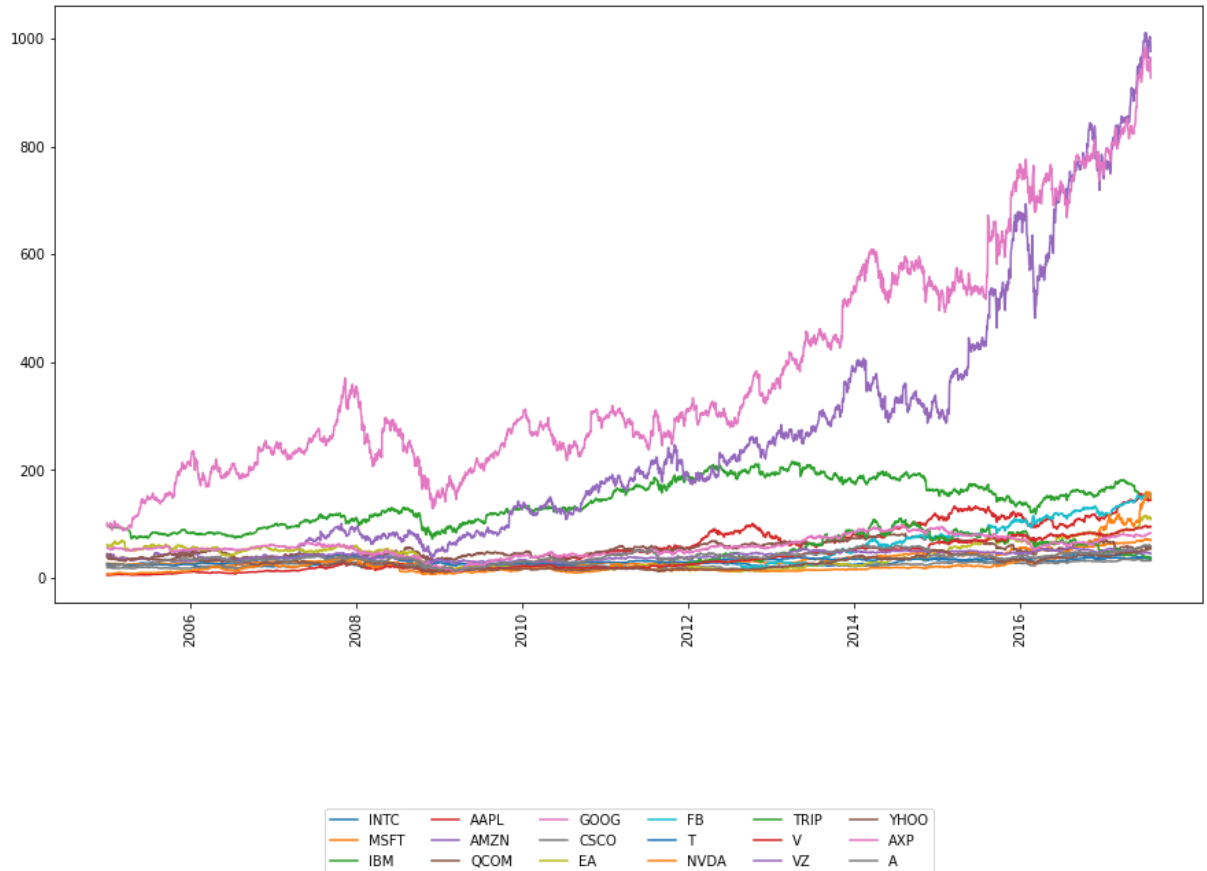
Ejercicio 3

Complete las sentencias necesarias para crear y representar lo más similar posible el siguiente gráfico de líneas. Ingrese el código completo en el espacio debajo.

```
1 import _____
2 import _____
3 _____ # transponemos el Dataframe prices
4 _____ # creamos el gráfico del tipo línea
5 # creamos un array para indicar la posición donde estarán las etiquetas
6 X = 250*(np.arange(6)*2+1)
7 plt.xticks(X,["2006","2008","2010","2012","2014","2016"],_____) # Rotamos etiquetas 90°
8 plt.legend(loc='_____', bbox_to_anchor=(0.5, -0.4), ncol=6) #centramos la Leyenda
9 _____ # Finalmente mostramos el gráfico
```



```
In [51]: import matplotlib.pyplot as plt
import numpy as np
transposePrices = prices.transpose()      # transponemos el Dataframe prices
transposePrices.plot(kind='line')          # creamos el gráfico del tipo línea
# creamos un array para indicar la posición donde estarán las etiquetas
X = 250*(np.arange(6)*2+1)
plt.xticks(X,["2006","2008","2010","2012","2014","2016"],rotation = 90) # Rotamos etiquetas 90°
plt.legend(loc='center', bbox_to_anchor=(0.5, -0.4),ncol=6)
plt.show()
```



Ejercicio 4

Realice un gráfico con subgráficos en cuatro secciones, donde cada sección corresponda a una empresa distinta. No se olvide de transponer previamente el dataframe antes de realizar los gráficos. Pueden ser gráficos iguales o diferentes. No se olvide de personalizar los gráficos (colores, formas, etc)

```
In [53]: import matplotlib.pyplot as plt

s1 = transposePrices['GOOG']
s2 = transposePrices['AMZN']
s3 = transposePrices['FB']
s4 = transposePrices['YHOO']
num_bins = 10

# usando el desempaquetado de tuplas para múltiples ejes
fig, ((ax1,ax2),(ax3,ax4)) = plt.subplots(2,2, figsize=(8,8))

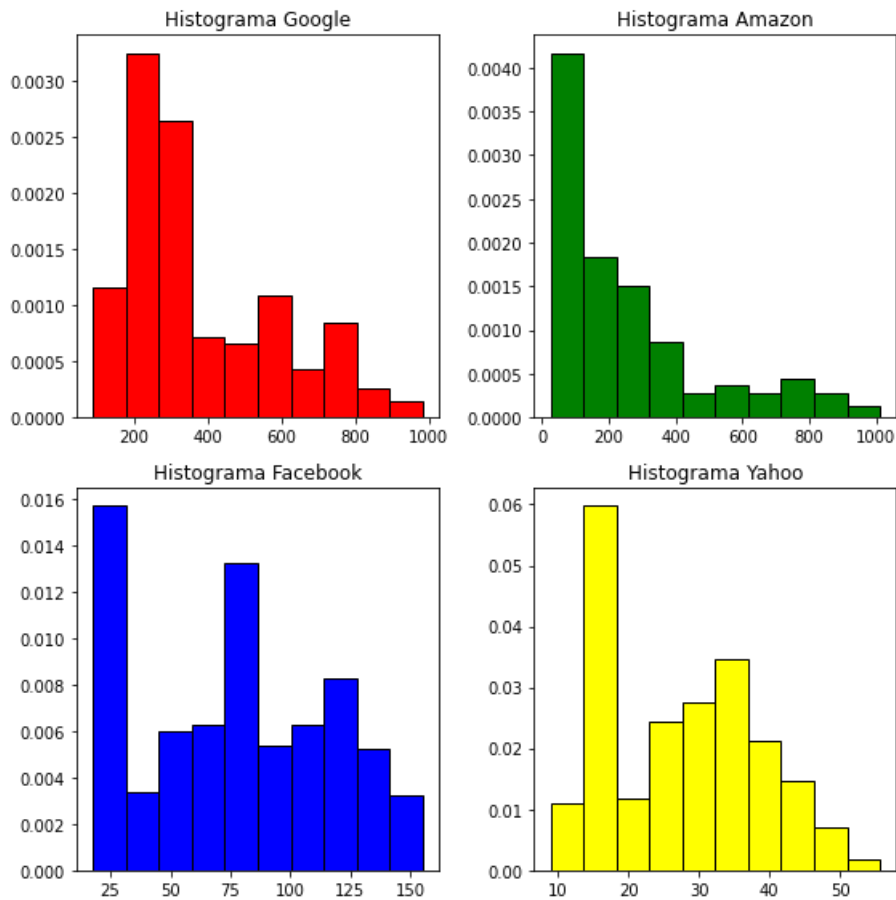
ax1.hist(s1, num_bins, density=1,edgecolor='black',color="red")
ax1.set_title('Histograma Google')

ax2.hist(s2, num_bins, density=1,edgecolor='black',color="green")
ax2.set_title('Histograma Amazon')

ax3.hist(s3, num_bins, density=1,edgecolor='black',color="blue")
ax3.set_title('Histograma Facebook')

ax4.hist(s4, num_bins, density=1,edgecolor='black',color="yellow")
ax4.set_title('Histograma Yahoo')

# Ajustar el espaciado para prevenir el recorte de la etiqueta y
fig.tight_layout()
plt.show()
```

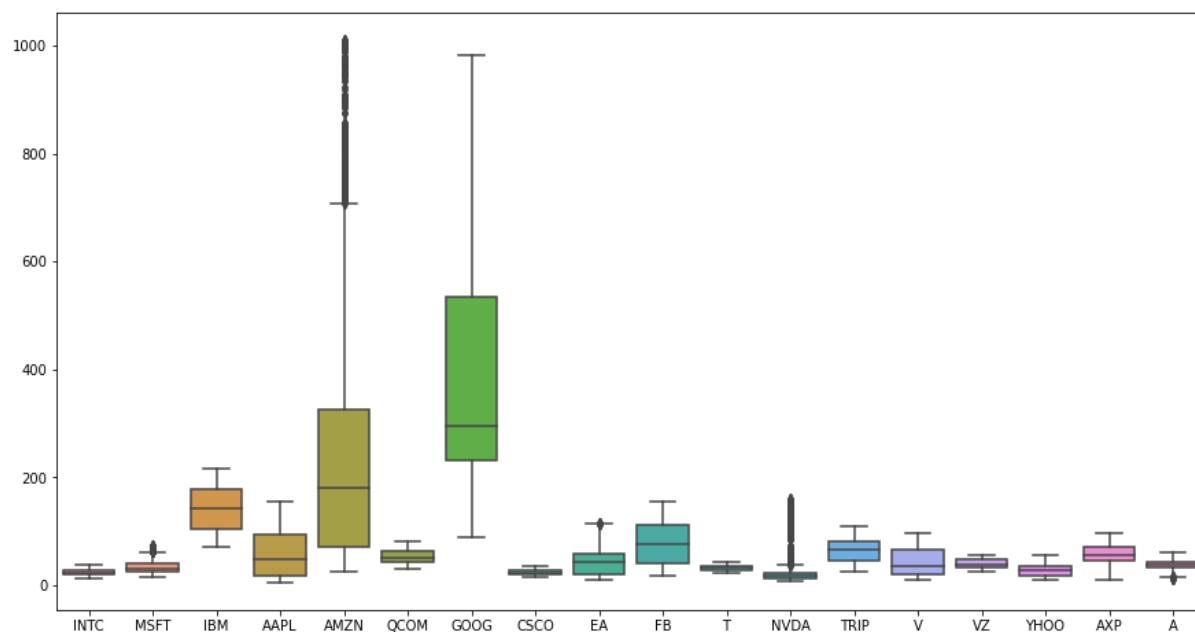


Ejercicio 5

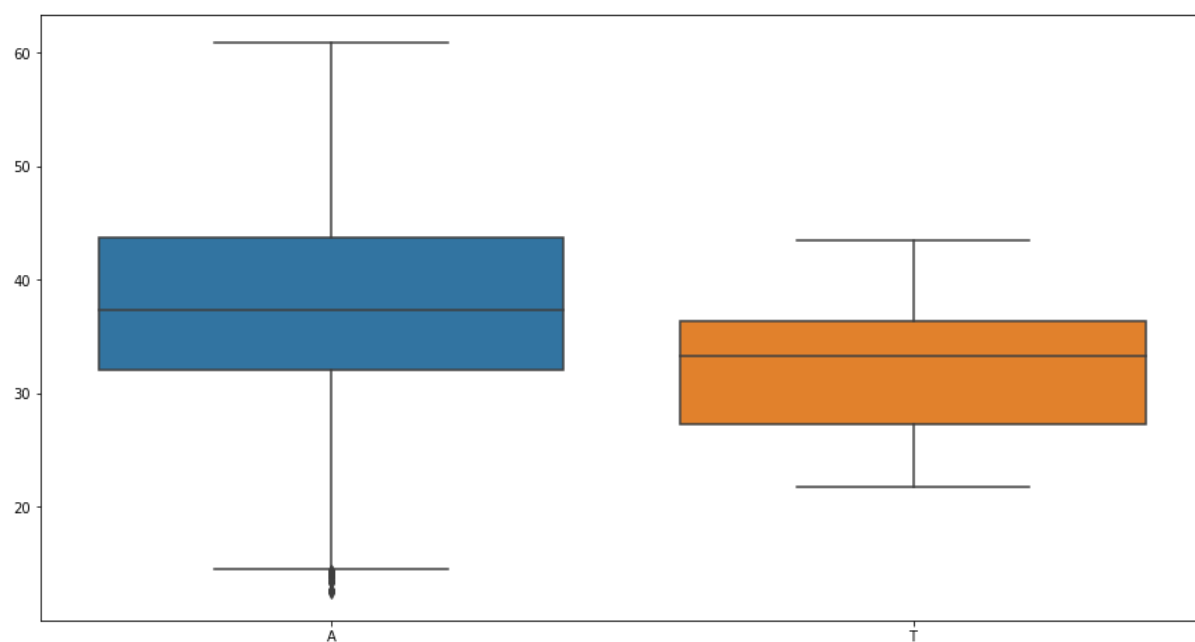
Realice los gráficos revisados en clase usando Series del dataframe cargado, y utilizando la librería Seaborn. No se olvide de transponer previamente el dataframe antes de realizar los gráficos.

```
In [54]: import seaborn as sns

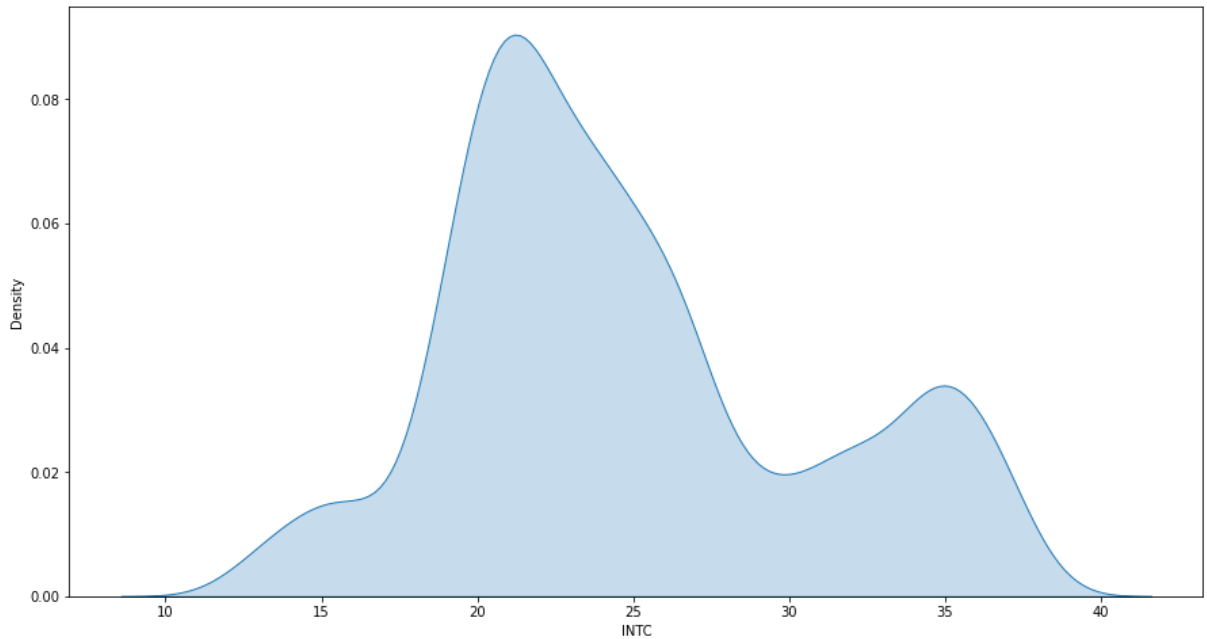
# vamos a graficar un par de ejemplos
sns.boxplot(data = transposePrices)
plt.show()
```



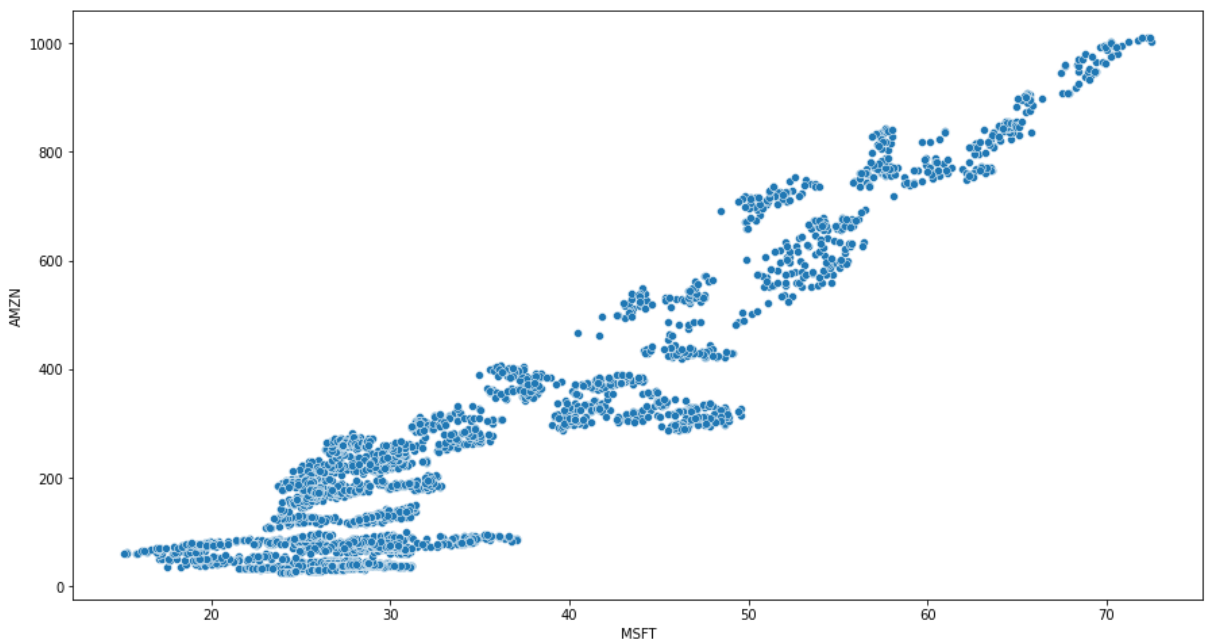
```
In [55]: sns.boxplot(data = transposePrices.loc[:,['A','T']])
plt.show()
```



```
In [56]: sns.kdeplot(data=nuevo_price['INTC'], shade=True)
plt.show()
```



```
In [57]: sns.scatterplot(x=nuevo_price.iloc[:,1], y=nuevo_price['AMZN'], data=nuevo_price)
plt.show()
```



Parte 4: Análisis Exploratorio de Datos

Continuando con la data de la sección anterior, responda las siguientes preguntas:

Ejercicio 1

¿Cuántos valores nulos hay en total en el dataframe prices?

```
In [58]: prices.isnull().sum().sum()
```

```
Out[58]: 4420
```

Hay 4420 valores nulos.

Ejercicio 2

Necesitamos el top 5 con las compañías que más valores nulos tienen en el intervalo ¿Cómo podrías calcularlo? ¿Qué empresa tiene más valores nulos, y cuantos valores nulos tiene?

```
In [59]: prices.transpose().isnull().sum().sort_values(ascending=False).head()
```

```
Out[59]: FB      1857
        TRIP    1746
        V       807
        EA        1
        YH00     1
        dtype: int64
```

La compañía con más valores nulos es Facebook (FB) con 1857 valores nulos

Ejercicio 3

Usando una expresión booleana y el operador .loc es posible extraer datos del dataframe. la expresión para obtener todos los valores de agosto de 2011 se muestra en la imagen. ¿Cuál era el valor de la acción de **Qualcomm (QCOM)** el 24 de junio de 2014?

```
transposePrices = prices.transpose()
flag = (transposePrices.index > '2011-08-01') & (transposePrices.index < '2011-09-01')
transposePrices.loc[flag,:]
```

```
In [60]: transposePrices = prices.transpose()
        flag = (transposePrices.index == '2014-06-24')
        transposePrices.loc[flag, 'QCOM']
```

```
Out[60]: 2014-06-24    78.78
        Name: QCOM, dtype: float64
```

El valor de la empresa Qualcomm el 24 de junio del 2014 era de 78.78

Ejercicio 4

¿Cuál es el valor del percentil 75% de **Microsoft (MSFT)**? Calcule el rango intercuartílico. Complemente estos resultados con un diagrama de cajas. Interprete los resultados.

```
In [61]: transposePrices = prices.transpose()

        #percentil 75 de MSFT
        q = 0.75
        transposePrices.loc[:, "MSFT"].quantile(q)
```

```
Out[61]: 40.34
```

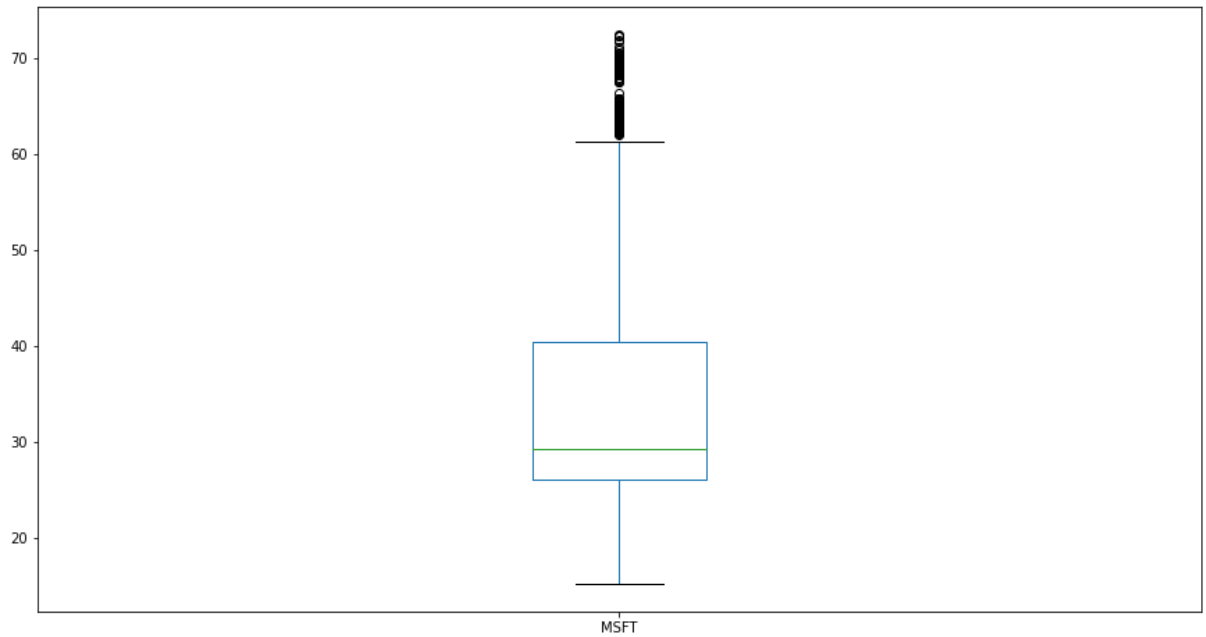
```
In [62]: #rango intercuartílico

        q2 = 0.25
        round(transposePrices.loc[:, "MSFT"].quantile(q) - transposePrices.loc[:, "MSFT"].quantile(q2), 2)
```

```
Out[62]: 14.28
```

```
In [63]: # diagrama de caja
```

```
import matplotlib.pyplot as plt
transposePrices = prices.transpose()
transposePrices.loc[:, "MSFT"].plot(kind='box')
plt.show()
```



Ejercicio 5

¿Cuál es el valor de promedio de **Apple (AAPL)**? ¿Y su desviación estándar? Realice un histograma de esta serie. Interprete los resultados.

```
In [64]: transposePrices = prices.transpose()

# promedio de las acciones de AAPL
round(transposePrices.loc[:, "AAPL"].mean(), 2)
```

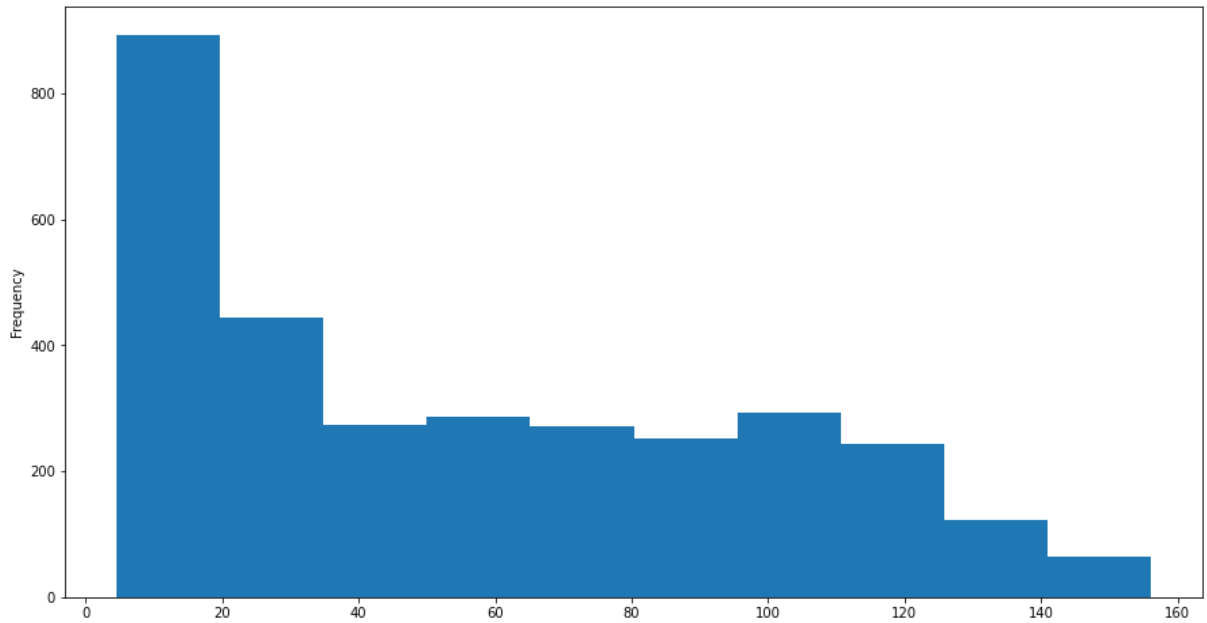
```
Out[64]: 56.07
```

```
In [65]: # desviación estándar de las acciones de AAPL
round(transposePrices.loc[:, "AAPL"].std(), 2)
```

```
Out[65]: 41.46
```

```
In [66]: # histograma

import matplotlib.pyplot as plt
transposePrices = prices.transpose()
transposePrices.loc[:, "AAPL"].plot(kind='hist')
plt.show()
```



Ejercicio 6

Se sospecha que cuando acciones de Facebook (FB) aumentan, las acciones de Yahoo (YHOO) también aumentan. Calculando la correlación entre estas 2 variables, ¿Es cierta la afirmación realizada? Complemente el resultado con un diagrama de dispersión.

```
In [67]: transposePrices = prices.transpose()
transposePrices.loc[:, ['FB', 'YHOO']].corr()
```

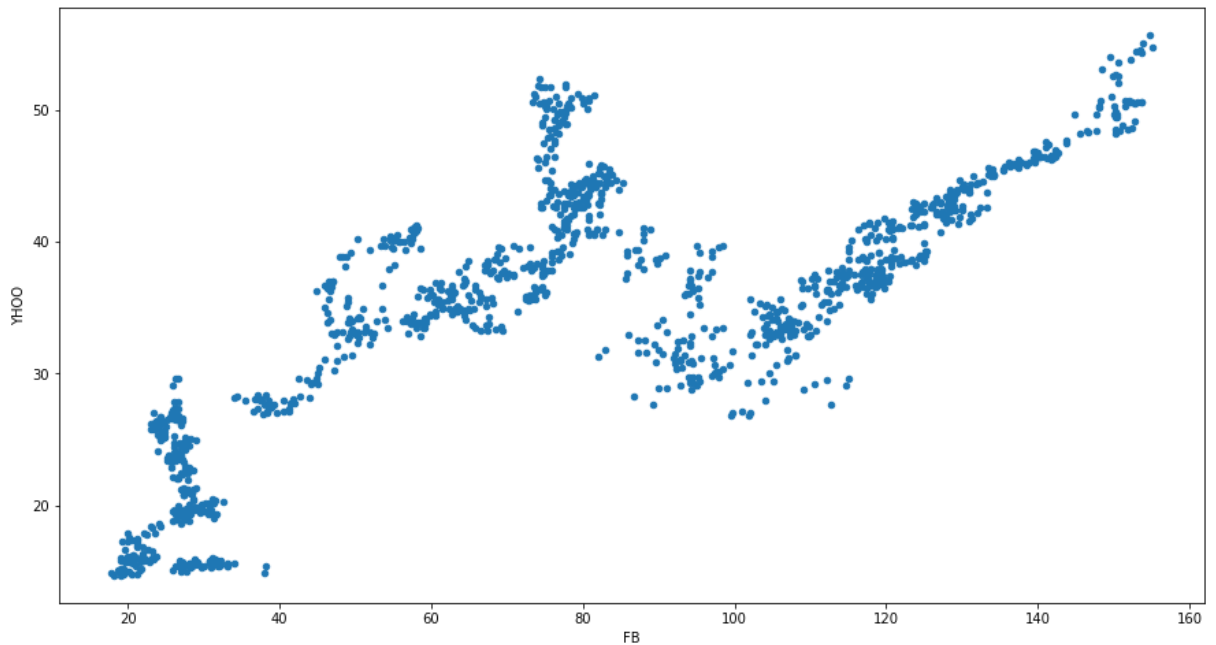
Out[67]:

	FB	YHOO
FB	1.000000	0.756946
YHOO	0.756946	1.000000

Es cierto, ya que la correlación es de 0.7569

```
In [68]: # histograma

import matplotlib.pyplot as plt
transposePrices.plot.scatter('FB', 'YHOO')
plt.show()
```



Ejercicio 7

Queremos obtener el listado de correlaciones de los valores de la empresa Microsoft (MSFT) con las demás empresas en orden ascendente. Complete los espacios en blanco para obtenerlo. Responda: ¿Cuál es la empresa cuyos valores tienen menos correlación con los de la empresa Microsoft? Complemente su resultado con una gráfica de matriz de correlaciones (diseño libre)

```
transposePrices = prices.transpose()
transposePrices.____()['MSFT'].sort_values(_____=True)
```

```
In [69]: transposePrices = prices.transpose()
transposePrices.corr()['MSFT'].sort_values(ascending=True)
```

```
Out[69]: TRIP      0.159151
         IBM       0.389841
         QCOM      0.512920
         A         0.631002
         T         0.646535
         AXP       0.684433
         EA        0.695391
         YHOO      0.727516
         CSCO      0.749834
         VZ        0.765453
         NVDA      0.797673
         AAPL      0.844110
         INTC      0.895888
         AMZN      0.933881
         GOOG      0.954344
         V         0.956676
         FB        0.968670
         MSFT      1.000000
         Name: MSFT, dtype: float64
```

La empresa cuyos valores tienen menos correlación con MSFT es TRIP

Ejercicio 8

El coeficiente de variación es una medida de dispersión relativa. Su ventaja principal es que no posee unidades. Se recomienda su uso para comparar la variabilidad entre dos o más conjunto de datos que tienen diferentes unidades de medida o que tienen promedios muy diferentes (como en nuestro caso). Las variables con valores más pequeños tienen menos variabilidad. Su fórmula se encuentra en la imagen (es la división entre la desviación estándar y el promedio). Si tomamos las 5 empresas con mayor valor promedio de las acciones (TRIP, GOOG, FB, AMZN, IBM) ¿Cuál de estas tiene la menor variabilidad relativa en los valores de sus acciones?

Para una muestra	Para una Población
$cv = \frac{S}{\bar{x}} \times 100$	$CV = \frac{\sigma}{\mu} \times 100$

```
In [70]: transpose = prices.transpose().loc[:,['FB','GOOG','IBM','AMZN','YHOO']]
(transpose.std()/transpose.mean()*100).sort_values(ascending=False)
```

```
Out[70]: AMZN      94.749393
GOOG      54.643533
FB        50.078985
YHOO      39.163996
IBM       28.782701
dtype: float64
```

La empresa que tiene menos variabilidad en los valores de sus acciones es IBM

Ejercicio 9

De acuerdo a un diagrama de cajas realizado a la data en la sección anterior, se observó que las empresas Amazon (AMZN) y Nvidia (NVDA) presentan la mayor cantidad de valores atípicos. En clase revisamos una función (llamada find_anomalies) para detectar los valores atípicos de una variable determinada. Esta función arroja una lista con los valores atípicos. Utilizando esta función, y cambiando el parámetro de corte a 3 desviaciones estándar, indique cuál de estas 2 empresas presenta más valores atípicos con el corte de 3 desviaciones, indicando la cantidad de outliers.

```
In [71]: anomalies = []

# Funcion ejemplo para detección de outliers
def find_anomalies(data):
    # Set upper and lower limit to 2 standard deviation
    data_std = data.std()
    data_mean = data.mean()
    anomaly_cut_off = data_std * 3 # podemos cambiar este corte
    lower_limit = data_mean - anomaly_cut_off
    upper_limit = data_mean + anomaly_cut_off
    print(lower_limit.iloc[0])
    print(upper_limit.iloc[0])

    # Generate outliers
    for index, row in data.iterrows():
        outlier = row # # obtener primer columna
        # print(outlier)
        if (outlier.iloc[0] > upper_limit.iloc[0]) or (outlier.iloc[0] < lower_limit.iloc[0]):
            anomalies.append(index)
    return anomalies
```

```
In [72]: transposePrices = prices.transpose()
print(find_anomalies(transposePrices[['AMZN']]))
print("Cantidad de outliers de la empresa AMAZON: ",len(anomalies))
print(find_anomalies(transposePrices[['NVDA']]))
print("Cantidad de outliers de la empresa NVIDIA: ",len(anomalies))

-457.2283552553007
953.5462674131614
['2017-05-12', '2017-05-15', '2017-05-16', '2017-05-18', '2017-05-19', '2017-05-22', '2017-05-23',
'2017-05-24', '2017-05-25', '2017-05-26', '2017-05-30', '2017-05-31', '2017-06-01', '2017-06-02',
'2017-06-05', '2017-06-06', '2017-06-07', '2017-06-08', '2017-06-09', '2017-06-12', '2017-06-13',
'2017-06-14', '2017-06-15', '2017-06-16', '2017-06-19', '2017-06-20', '2017-06-21', '2017-06-22',
'2017-06-23', '2017-06-26', '2017-06-27']
Cantidad de outliers de la empresa AMAZON: 31
-45.758038096026795
92.78925388215019
['2017-05-12', '2017-05-15', '2017-05-16', '2017-05-18', '2017-05-19', '2017-05-22', '2017-05-23',
'2017-05-24', '2017-05-25', '2017-05-26', '2017-05-30', '2017-05-31', '2017-06-01', '2017-06-02',
'2017-06-05', '2017-06-06', '2017-06-07', '2017-06-08', '2017-06-09', '2017-06-12', '2017-06-13',
'2017-06-14', '2017-06-15', '2017-06-16', '2017-06-19', '2017-06-20', '2017-06-21', '2017-06-22',
'2017-06-23', '2017-06-26', '2017-06-27', '2016-11-18', '2016-11-21', '2016-11-22', '2016-11-23',
'2016-11-25', '2016-11-28', '2016-11-29', '2016-12-06', '2016-12-07', '2016-12-08', '2016-12-14',
'2016-12-15', '2016-12-16', '2016-12-19', '2016-12-20', '2016-12-21', '2016-12-22', '2016-12-23',
'2016-12-27', '2016-12-28', '2016-12-29', '2016-12-30', '2017-01-03', '2017-01-04', '2017-01-05',
'2017-01-06', '2017-01-09', '2017-01-10', '2017-01-11', '2017-01-12', '2017-01-13', '2017-01-17',
'2017-01-18', '2017-01-19', '2017-01-20', '2017-01-23', '2017-01-24', '2017-01-25', '2017-01-26',
'2017-01-27', '2017-01-30', '2017-01-31', '2017-02-01', '2017-02-02', '2017-02-03', '2017-02-06',
'2017-02-07', '2017-02-08', '2017-02-09', '2017-02-10', '2017-02-13', '2017-02-14', '2017-02-15',
'2017-02-16', '2017-02-17', '2017-02-21', '2017-02-22', '2017-02-23', '2017-02-24', '2017-02-27',
'2017-02-28', '2017-03-01', '2017-03-02', '2017-03-03', '2017-03-06', '2017-03-07', '2017-03-08',
'2017-03-09', '2017-03-10', '2017-03-13', '2017-03-14', '2017-03-15', '2017-03-16', '2017-03-17',
'2017-03-20', '2017-03-21', '2017-03-22', '2017-03-23', '2017-03-24', '2017-03-27', '2017-03-28',
'2017-03-29', '2017-03-30', '2017-03-31', '2017-04-03', '2017-04-04', '2017-04-05', '2017-04-06',
'2017-04-07', '2017-04-10', '2017-04-11', '2017-04-12', '2017-04-13', '2017-04-17', '2017-04-18',
'2017-04-19', '2017-04-20', '2017-04-21', '2017-04-24', '2017-04-25', '2017-04-26', '2017-04-27',
'2017-04-28', '2017-05-01', '2017-05-02', '2017-05-03', '2017-05-04', '2017-05-05', '2017-05-08',
'2017-05-09', '2017-05-10', '2017-05-11', '2017-05-12', '2017-05-15', '2017-05-16', '2017-05-17',
'2017-05-18', '2017-05-19', '2017-05-22', '2017-05-23', '2017-05-24', '2017-05-25', '2017-05-26',
'2017-05-30', '2017-05-31', '2017-06-01', '2017-06-02', '2017-06-05', '2017-06-06', '2017-06-07',
'2017-06-08', '2017-06-09', '2017-06-12', '2017-06-13', '2017-06-14', '2017-06-15', '2017-06-16',
'2017-06-19', '2017-06-20', '2017-06-21', '2017-06-22', '2017-06-23', '2017-06-26', '2017-06-27']
Cantidad de outliers de la empresa NVIDIA: 175
```

La empresa NVIDIA es la que tiene mayor cantidad de outliers, con la cantidad de 175.

Parte 5: Variables aleatorias y distribuciones muestrales

```
In [73]: # importando paquetes necesarios para esta sección
import matplotlib.pyplot as plt
import numpy as np # importando numpy
import pandas as pd # importando pandas
import scipy.stats as ss # importando scipy.stats
```

Ejercicio 1

El jefe de recursos humanos de una empresa realiza un test de diez ítems a los aspirantes a un puesto, teniendo en cada ítems cuatro posibles respuestas, de las que sólo una es correcta. Suponiendo que los aspirantes teniendo la misma probabilidad de responder. Se pide hallar las probabilidades para el aspirante:

- Conteste todos los ítems mal
- Conteste al menos cuatro ítems bien
- Conteste entre cuatro y seis ítems bien

```
In [74]: #cdf(x) - Función de distribución F(X)
#sf(x) = 1 - cdf(x)
#pmf(x) - Función de probabilidad f(x) (distribuciones discretas)
#pdf(x) - Función de densidad f(x) (distribuciones continuas)
#ppf(x) - Función inversa a cdf(x). Nos permite obtener el valor correspondiente a una probabilidad d.

X = ss.binom(10,0.25)
pr =round(X.pmf(0),4)*100
print("Probabilidad que conteste todos los ítems mal es del: "+str(pr)+"%")
pr1=round(X.sf(3),4)*100
print("Probabilidad que conteste al menos cuatro ítems bien es del: "+str(pr1)+"%")
pr2=round((X.pmf(4)+X.pmf(5)+X.pmf(6)),2)*100
print("Probabilidad que conteste entre cuatro y seis ítems bien es del: "+str(pr2)+"%")
```

Probabilidad que conteste todos los ítems mal es del: 5.63%
 Probabilidad que conteste al menos cuatro ítems bien es del: 22.41%
 Probabilidad que conteste entre cuatro y seis ítems bien es del: 22.0%

Ejercicio 2

Para el tiempo empleado, en horas, en hacer un determinado producto sigue una distribución $N(10, 2)$. Se pide la probabilidad de que ese producto se tarde en hacer:

- Menos de 7 horas
- Entre 8 y 13 horas

```
In [75]: X = ss.norm(10,2)
pr1=round(1-X.sf(7),4)*100
print("Probabilidad de que el producto se tarde en hacer Menos de 7 horas es del: "+str(pr1)+"%")

pr2=round((X.cdf(13) - X.cdf(8)),4)*100
print("Probabilidad de que el producto se tarde en hacer Entre 8 y 13 horas es del: "+str(pr2)+"%")
)
```

Probabilidad de que el producto se tarde en hacer Menos de 7 horas es del: 6.68%
 Probabilidad de que el producto se tarde en hacer Entre 8 y 13 horas es del: 77.45%

Ejercicio 3

Una compañía aérea observa que el número de componentes que fallan antes de cumplir 100 horas de funcionamiento es una variable aleatoria de Poisson. Si el número promedio de fallos es ocho. Se pide:

- ¿Cuál es la probabilidad de que falle un componente en 25 horas?
- ¿Cuál es la probabilidad de que fallen menos de dos componente en 50 horas?

```
In [76]: # el promedio de fallos es 8 ara 100 horas
# por lo tanto el número de fallos es 4 para 50 horas
# y el número de fallos es 2 para 25 horas

X = ss.poisson(2)
pr1=round(X.pmf(1),4)*100
print("Probabilidad de que falle un componente en 25 horas es del: "+str(pr1)+"%")

X = ss.poisson(4)
pr2=round(X.cdf(1),4)*100
print("Probabilidad de que falle menos de dos componente en 50 horas es del: "+str(pr2)+"%")
```

Probabilidad de que falle un componente en 25 horas es del: 27.07%
 Probabilidad de que falle menos de dos componente en 50 horas es del: 9.16%

Ejercicio 4

En una población de mujeres, las puntuaciones de un test de ansiedad riesgo siguen una distribución normal $N(25,10)$. Al clasificar la población en cuatro grupos de igual tamaño, ¿cuales serán las puntuaciones que delimiten estos grupos?

```
In [77]: X = ss.norm(25,10)

q1=round(X.ppf(q=0.25),1)
q2=round(X.ppf(q=0.5),1)
q3=round(X.ppf(q=0.75),1)
print("El primer grupo tendrá puntuaciones inferiores o iguales a: ",q1)
print("El segundo grupo tendrá puntuaciones entre: ",q1,"y",q2)
print("El tercer grupo tendrá puntuaciones entre: ",q2,"y",q3)
print("El último grupo tendrá puntuaciones superiores a: ",q3)
```

El primer grupo tendrá puntuaciones inferiores o iguales a: 18.3
 El segundo grupo tendrá puntuaciones entre: 18.3 y 25.0
 El tercer grupo tendrá puntuaciones entre: 25.0 y 31.7
 El último grupo tendrá puntuaciones superiores a: 31.7

Ejercicio 5

Un pasajero opta por una compañía aérea con probabilidad 0,5. En un grupo de 400 pasajeros potenciales, la compañía vende billetes a cualquiera que se lo solicita, sabiendo que la capacidad de su avión es de 230 pasajeros. Se pide la probabilidad de que la compañía tenga overbooking, es decir, que un pasajero no tenga asiento.

```
In [78]: # Forma 1: directamente con al binomial

bi = ss.binom(400,0.5)
prob=round(bi.sf(230),6)*100
print("Y la probabilidad de que la compañía tenga overbooking es del: " ,prob,"%")
```

Y la probabilidad de que la compañía tenga overbooking es del: 0.1124 %

```
In [79]: # Forma 2: aproximación de La binomial a La normal

n,p=400,0.5
media=n*p
desviacion=np.sqrt(n*p*(1-p))

X = ss.norm(media, desviacion)
prob=round(X.sf(230),6)*100

print("La variable X = pasajeros que optan por esa compañía, tiene distribucion binomial X~ b(400, 0,5)")
print("La distribución binomial se puede aproximar a una distribución normal, por el teorema del l ímite central")
print("Por lo tanto, la distribucion normal sería: N ~ (",media,",",desviacion,")")
print("Y la probabilidad sería del: " ,prob,"%")
```

La variable X = pasajeros que optan por esa compañía, tiene distribucion binomial X~ b(400, 0,5)
 La distribución binomial se puede aproximar a una distribución normal, por el teorema del límite c
 entral
 Por lo tanto, la distribucion normal sería: N ~ (200.0 , 10.0)
 Y la probabilidad sería del: 0.135 %

Ejercicio 6

El número de ventas diarias de un quiosco de periódicos se distribuye con media 30 y varianza 2. Determinar:

- Probabilidad de que en un día se vendan entre 13 y 31 periódicos
- Determinar el número de periódicos que se venden en el 90% de las ocasiones

```
In [80]: X = ss.norm(30,np.sqrt(2))
pr13=round((X.cdf(31) - X.cdf(13)),4)*100
print("Probabilidad de que en un día se vendan entre 13 y 31 periódicos es del: "+str(pr13)+"%")

pr14=round(ss.norm.ppf(q=0.9,loc=30,scale=np.sqrt(2)),2)

print("El número de periódicos que se venden en el 90% de las ocasiones es de: "+str(pr14),"period  

  icos")
```

Probabilidad de que en un día se vendan entre 13 y 31 periódicos es del: 76.02%
 El número de periódicos que se venden en el 90% de las ocasiones es de: 31.81 periodicos

Ejercicio 7

Un banco vende un promedio de 20 seguros al día, suponiendo que el número de seguros vendidos sigue una distribución de Poisson. Se pide:

- Probabilidad de que se venda 18 seguros en un día
- Probabilidad de que se vendan más de 145 seguros en una semana

```
In [81]: X = ss.poisson(20)
pr15=round(X.pmf(18),4)*100
print("Probabilidad de que se venda 18 seguros en un día es del: "+str(pr15)+"%", "\n")

# 20seg 1 dia
# ? 7 dia
# entonces landa es igual a 140
print("El lambda de la distribucion poisson es grande, por tal motivo la distribucion se aproxima a la normal")
print("se aproxima a una distribución normal  $X \sim N(140, \text{raiz}(140))$ ")

X = ss.norm(140,np.sqrt(140))
pr16=round(X.sf(145),2)*100
print("Probabilidad de que se vendan más de 145 seguros en una semana es del: ",pr16,"%")
```

Probabilidad de que se venda 18 seguros en un día es del: 8.44%

El lambda de la distribucion poisson es grande, por tal motivo la distribucion se aproxima a la normal
se aproxima a una distribución normal $X \sim N(140, \text{raiz}(140))$
Probabilidad de que se vendan más de 145 seguros en una semana es del: 34.0 %

Ejercicio 8

El peso de un determinado tipo de naranja fluctúa normalmente con media 180 gramos y desviación típica 20 gramos. Una bolsa de llena con 20 manzanas seleccionadas al azar. ¿Cuál es la probabilidad de que el peso total de la bolsa sea inferior a 3.5 kilos?

```
In [82]: print("Como son 15 naranjas,entonces la distribucion normal sera: ")
print("X~ N(15*180, raiz(15*20*20)) --> X~ N(2700, raiz(6000))")

X = ss.norm(2700,np.sqrt(6000))
pr17=round(1-X.sf(3500),4)*100
print("Probabilidad de que el peso total de la bolsa sea inferior a 3.5 kilos es del: ",pr17,"%")
```

Como son 15 naranjas,entonces la distribucion normal sera:
 $X \sim N(15 \cdot 180, \text{raiz}(15 \cdot 20 \cdot 20)) \rightarrow X \sim N(2700, \text{raiz}(6000))$
Probabilidad de que el peso total de la bolsa sea inferior a 3.5 kilos es del: 100.0 %

Ejercicio 9

Un lote de partes para ensamblar en una empresa está formado por 100 elementos del proveedor A y 200 elementos del proveedor B. Se selecciona una muestra de 4 partes al azar sin reemplazo de las 300 para una revisión de calidad.

- Calcular la probabilidad de que todas las 4 partes de la muestra sean del proveedor A.
- Calcular la probabilidad de que dos o más de las partes sean del proveedor A.

Sugerencia: utilice la distribución hipergeométrica

```
In [83]: X = ss.hypergeom(300,4,100)
pr18=round(X.pmf(4),3)*100
print("Probabilidad de que todas las 4 partes de la muestra sean del proveedor A es del: ",pr18,"%")

pr19=(round(X.pmf(4),3)*100+round(X.pmf(3),3)*100+round(X.pmf(2),3)*100)
print("Probabilidad de que dos o más de las partes sean del proveedor A es del: ",pr19,"%")
```

Probabilidad de que todas las 4 partes de la muestra sean del proveedor A es del: 1.2 %
Probabilidad de que dos o más de las partes sean del proveedor A es del: 40.8 %

Parte 6: Inferencia Estadística

Ejercicio 1

Se toma una muestra de 60 individuos de una población que se sabe tiene una desviación estándar de 1,4. Se encuentra que la media de esta muestra es de 6,2. Construya un intervalo de confianza del 95% de la media poblacional. Interprete el intervalo.

```
In [84]: tstar_95 = 1.96
media=6.2
sd =1.4
n = 60

se = sd/np.sqrt(n)
se

lcb = round((media - tstar_95 * se),2)
ucb = round((media + tstar_95 * se),2)
(lcb, ucb)
print("Hay un 95% de confianza que la media poblacional esté entre:")
print(lcb, "-", ucb,)
```

Hay un 95% de confianza que la media poblacional esté entre:
5.85 - 6.55

Ejercicio 2

Se llevó a cabo una encuesta de mercado para calcular la proporción de amas de casa que reconocerían el nombre de la marca de un limpiador a partir de la forma y color del envase. De las 1400 amas de casa de la muestra, 420 identificaron la marca por su nombre.

- Calcule el valor de la proporción de la población.
- Construya el intervalo de confianza de 99% de la proporción poblacional.

```
In [85]: tstar_99 = 2.58
p = 420/1400
p1=round(420/1400,2)*100
n = 1400

se = np.sqrt((p * (1 - p))/n)
se
lcb = round((p - tstar_99 * se),3)*100
ucb = round((p + tstar_99 * se),3)*100
(lcb, ucb)

print("El valor de la propoción de amas de casas es de: ",p1,"%")
print("Hay un 99% de confianza que la proporción de amas de casa que reconocerían el limpiador est á entre:")
print(lcb,"%", "-", ucb,"%")
```

El valor de la propoción de amas de casas es de: 30.0 %
Hay un 99% de confianza que la proporción de amas de casa que reconocerían el limpiador está entre:
26.8 % - 33.2 %

Ejercicio 3

De una muestra de 18 gasolineras REPSOL tomadas en la ciudad de Lima, se encontró que el precio promedio de un galón de gasolina sin plomo es de S/ 3,17; con una desviación estándar de S/ 0,08 por galón. Halle el intervalo de confianza al 90% para el valor real del precio medio de la gasolina sin plomo por galón.

```
In [86]: n=18
media=3.17
sd=0.08
tstar_90=1.65

se = sd/np.sqrt(n)
se

lcb = round((media - tstar_90 * se),2)
ucb = round((media + tstar_90 * se),2)
(lcb, ucb)
print("Hay un 90% de confianza que el precio promedio de la gasolina esté entre los valores en sol
es de:")
print(lcb,"-",ucb,)
```

Hay un 90% de confianza que el precio promedio de la gasolina esté entre los valores en soles de:
3.14 - 3.2

Ejercicio 4

Se selecciona una muestra de 36 observaciones de una población normal. La media de la muestra es 21, y la desviación estándar de la población, 5. Lleve a cabo la prueba de hipótesis para confirmar de que el promedio poblacional es igual a 20 (use un nivel de significancia de 0.05)

```
In [87]: from scipy import stats
n=36
media=21
sd=5
x=20
alpha=0.05
z=(x-media)/(sd/np.sqrt(n))
p = 1 - stats.norm.cdf(z)
if p < alpha:
    print("p-valor =",p,"; Se rechaza la H0: Promedio = 20")
else:
    print("p-valor =",p,"; Se acepta la H0: Promedio = 20")
```

p-valor = 0.8849303297782918 ; Se acepta la H0: Promedio = 20

Ejercicio 5

Considere una muestra de 40 observaciones de una población con una desviación estándar de la población de 5. La media muestral es 102. Otra muestra de 50 observaciones de una segunda población tiene una desviación estándar de la población de 6. La media muestral es 99. Realice la prueba de hipótesis de que la media de ambas poblaciones son iguales, con el nivel de significancia de 0.04.

```
In [88]: u1 = 102
u2 = 99

s=np.sqrt((5**2+6**2)/2)

pval=(u1-u2)/(s*np.sqrt(2/90))

print(pval)

if pval<0.04:
    print("pval is ",pval,"Se rechaza la hipótesis nula, los promedios muestrales son estadísticam
ente diferentes")
else:
    print("pval is ",pval,"No se rechaza la hipótesis nula, los promedios muestrales no son estadí
sticamente diferentes")
```

3.6439934858051215

pval is 3.6439934858051215 No se rechaza la hipótesis nula, los promedios muestrales no son estadísticamente diferentes

Ejercicio 6

Desde hace algún tiempo las aerolíneas han reducido sus servicios, como alimentos y bocadillos durante sus vuelos, y empezaron a cobrar un precio adicional por algunos de ellos, como llevar sobrepeso de equipaje, cambios de vuelo de último momento y por mascotas que viajan en la cabina. Sin embargo, aún están muy preocupadas por el servicio que ofrecen. Hace poco un grupo de cuatro aerolíneas contrató a Brunner Marketing Research, Inc., para encuestar a sus pasajeros sobre la adquisición de boletos, abordaje, servicio durante el vuelo, manejo del equipaje, comunicación del piloto, etc. Hicieron 25 preguntas con diversas respuestas posibles: excelente, bueno, regular o deficiente. Una respuesta de excelente tiene una calificación de 4, bueno 3, regular 2 y deficiente 1. Estas respuestas se sumaron, de modo que la calificación final fue una indicación de la satisfacción con el vuelo. Entre mayor la calificación, mayor el nivel de satisfacción con el servicio. La calificación mayor posible fue 100. Brunner seleccionó y estudió al azar pasajeros de las cuatro aerolíneas. A continuación, se muestra la información. ¿Hay alguna diferencia entre los niveles de satisfacción medios con respecto a las cuatro aerolíneas? Use el nivel de significancia de 0.01.

Northern	WTA	Pocono	Branson
94	75	70	68
90	68	73	70
85	77	76	72
80	83	78	65
	88	80	74
		68	65
		65	

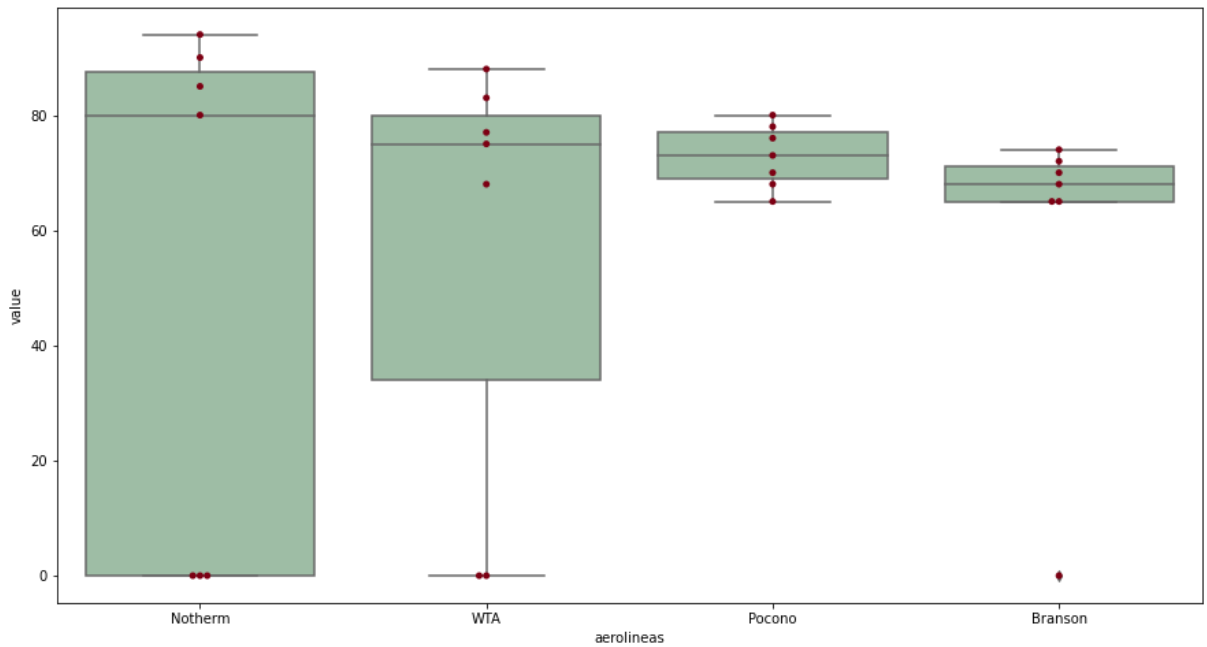
```
In [89]: import numpy as np
import seaborn as sns
import pandas as pd

#creando la data en un dataframe para poderlo graficar
set1={'Notherm':[94,90,85,80,0,0,0], 'WTA':[75,68,77,83,88,0,0],
      'Pocono':[70,73,76,78,80,68,65], 'Branson':[68,70,72,65,74,65,0]}
df1 = pd.DataFrame(set1)
df_melt = pd.melt(df1.reset_index(), id_vars=['index'], value_vars=['Notherm', 'WTA', 'Pocono', 'Branson'])
df_melt.columns = ['index', 'aerolineas', 'value']
df1
```

Out[89]:

	Notherm	WTA	Pocono	Branson
0	94	75	70	68
1	90	68	73	70
2	85	77	76	72
3	80	83	78	65
4	0	88	80	74
5	0	0	68	65
6	0	0	65	0

```
In [90]: ax = sns.boxplot(x='aerolineas', y='value', data=df_melt, color='#99c2a2')
ax = sns.swarmplot(x="aerolineas", y="value", data=df_melt, color='#7d0013')
plt.show()
```



```
In [92]: # por el gráfico parece que hay diferencia entre los niveles promedio de satisfacción de las 4 aerolíneas

NorThem={'NorThem': pd.Series([94, 90, 85,80])}
NorThem = pd.DataFrame(NorThem)
WTA={'WTA': pd.Series([75, 68, 77, 83,88])}
WTA = pd.DataFrame(WTA)
Pocono={'Pocono': pd.Series([70,73,76,78,80,68,65])}
Pocono = pd.DataFrame(Pocono)
Branson={'Branson': pd.Series([68,70,72,65,74,65])}
Branson = pd.DataFrame(Branson)

pval = ss.f_oneway(NorThem, WTA, Pocono,Branson)[1]

print(pval.round(5))
print('Se rechaza la hipótesis nula. Hay diferencia entre los niveles promedio de satisfacción entre las aerolíneas.')
```

[0.00074]

Se rechaza la hipótesis nula. Hay diferencia entre los niveles promedio de satisfacción entre las aerolíneas.

Ejercicio 7

Se da la siguiente situación de prueba de hipótesis: $H_0: p = 0.50$ y $H_1: p \neq 0.50$. El nivel de significancia es 0.05, y el tamaño de la muestra es 9. b. Hubo cinco éxitos. ¿Cuál es su decisión respecto de la hipótesis nula?

```
In [93]: from scipy.stats import binom_test

pval_5_exit = binom_test(5,n=9,p=0.5)
print(pval_5_exit)
print('No podemos rechazar la hipótesis nula. El resultado se encuentra dentro de lo esperado.')
```

1.0

No podemos rechazar la hipótesis nula. El resultado se encuentra dentro de lo esperado.

Ejercicio 8

Para estudiar la dependencia entre la práctica de algún deporte y la depresión, se seleccionó una muestra al azar de 100 jóvenes, obteniendo los siguientes resultados:

	Sin depresión	Con depresión
Deportista	38	9
No deportista	31	22

Determinar si existe independencia entre la actividad del sujeto y su estado de ánimo (use un nivel de 5% de significancia)

Sugerencia: la hipótesis nula es la independencia de las variables

```
In [94]: from scipy.stats import chi2_contingency

X = [[38,9],
     [31,22]]

chi2, pval, dof, expected = chi2_contingency(X)
print(pval.round(5))
print('P-valor es menor a 0.05')
print('No podemos rechazar la hipótesis nula. No existe una relación significativa entre las variables.')
```

0.02806
P-valor es menor a 0.05
No podemos rechazar la hipótesis nula. No existe una relación significativa entre las variables.